Lab 8: Memory Management in xv6

Raj Pratap Singh Gurjar | 2401MC21 | Operating Systems Lab
Base: xv6-public commit eeb7b415dbcb12cc362d0783e41c3d1f44066b17

I added four memory-management features to xv6: virtual-to-physical translation, memory statistics, a page-table dump, and lazy heap allocation. The patches apply in question order to the base commit above.

Q1 — va2pa

The new system call walks the current process page table and returns the physical address for a present user page, including the byte offset. It returns -1 for an unmapped page or a mapping without user permission. *vatest* covers a global, stack local, heap address, unmapped address, and the same global virtual address in the parent and child after fork. The parent and child map that address to different physical pages.

Q2 — getmeminfo

The call reports total physical pages, free pages, the caller's size in pages, and page-table pages. The free-page count is maintained alongside the allocator's free list under its lock. In *memtest*, touching 16 pages reduces the free count by 16; after the child exits, the count returns to its earlier value.

Q3 — vmprint

The dump lists user page-directory entries and their present PTEs with virtual address, physical address, and P/W/U flags. Kernel entries are counted in a summary line. *vmprinttest* confirms the guard page is mapped without user permission, then checks the mappings after growing and shrinking the heap.

Q4 — lazy heap allocation

A positive *sbrk* records the new size without allocating data frames. A page fault inside that range allocates and zeroes a page; faults outside the range and faults on present protected pages remain fatal. Shrinking skips untouched pages, and oversized growth is rejected. *copyuvm* skips absent pages so fork works over a mostly untouched heap. Kernel writes into lazy user buffers are handled through *copyout*, and kernel reads fault in eligible user pages through the page-fault handler.

Checks from the captured QEMU run

- The 32 MB heap reservation used no free pages. Touching 32 spaced pages used 39 pages: 32 data pages and 7 page-table pages.
- Shrinking across untouched pages succeeded, and an oversized *sbrk* was rejected.
- *getmeminfo* and *read* worked with untouched output buffers; a pipe write from an untouched buffer transferred 16 bytes.
- A forked child faulted in a zero-filled page; after it exited, the parent's page remained zero and free pages returned to the pre-fork count.
- An invalid address killed only the child. The stock *forktest* ended with "fork test OK."

The four patches are in diffs/, the user programs are in tests/, and output_log.txt contains the QEMU serial output used for the terminal captures in screenshots/.